
TIER: Trajectory-Invariant Execution Rewards for Multi-Step Tool Composition

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Tool use enables large language models to solve complex tasks through sequences
2 of API calls, yet existing reinforcement learning approaches fail to scale to multi-
3 step composition settings. Outcome-based rewards provide only sparse feedback,
4 while trajectory-supervised rewards depend on annotated reference solutions, pe-
5 nalizing valid alternatives and limiting scalability. We propose **TIER**: Trajectory-
6 Invariant Execution Rewards, a reward framework that derives supervision directly
7 from function schemas and runtime execution, rather than from reference trajec-
8 tories. The reward decomposes into format validity, schema adherence, execution
9 success, and answer correctness, providing dense, interpretable sequence-level
10 feedback derived from fine-grained verification of individual steps of tool use.
11 This design allows any valid execution path to receive credit, naturally supporting
12 multiple solution strategies and adapting to evolving tool interfaces. On Depth-
13 Bench, a compositional benchmark stratified by depth (1 to 6 steps), TIER achieves
14 >90% accuracy across steps, where trajectory-supervised rewards collapse beyond
15 step-4. We further demonstrate consistent gains on benchmarks like BFCL v3
16 and NestFUL. Ablation studies confirm that all reward components are necessary,
17 highlighting the importance of multi-level supervision for compositional reasoning.

18 1 Introduction

19 Large language models (LLMs) have shown strong capabilities in tool use, enabling interaction with
20 external environments through structured API calls [Schick et al., 2023, Qin et al., 2023]. Recent
21 reinforcement learning (RL) approaches further improve tool-use performance [Qian et al., 2025,
22 Guo et al., 2025], but they primarily target single-step tool invocation or aggregate success metrics.
23 In contrast, tasks that require composing multiple tool calls remain challenging, with performance
24 degrading rapidly as composition depth increases.

25 Tool composition refers to tasks where solutions require multiple interdependent tool calls. For
26 example, answering "*What is the weather at the destination of my next flight?*" requires first retrieving
27 the flight information, extracting its destination, and only then querying the weather. We refer to the
28 longest chain of tools required for a task as its *depth*. We demonstrate that under simple outcome
29 rewards, accuracy can drop from over 94% on 1-step, to below 2% at 2-step, and even 0% beyond
30 4-step compositions (Table 1). This limits the deployment of LLM-based agents in real-world settings.

31 A key challenge lies in reward design. Outcome-based reinforcement learning with verifiable rewards
32 (RLVR) provides only a sparse, binary signal upon task completion, making credit assignment for
33 intermediate decisions such as tool selection, argument specification, and execution order, diffi-
34 cult [Shao et al., 2024, Guo et al., 2025]. Recent works introduce more fine-grained supervision by
35 comparing model outputs to human-curated reference trajectories [Qian et al., 2025, Zeng et al., 2025].
36 However, trajectory-based rewards penalize valid alternative solutions, require costly annotation, and

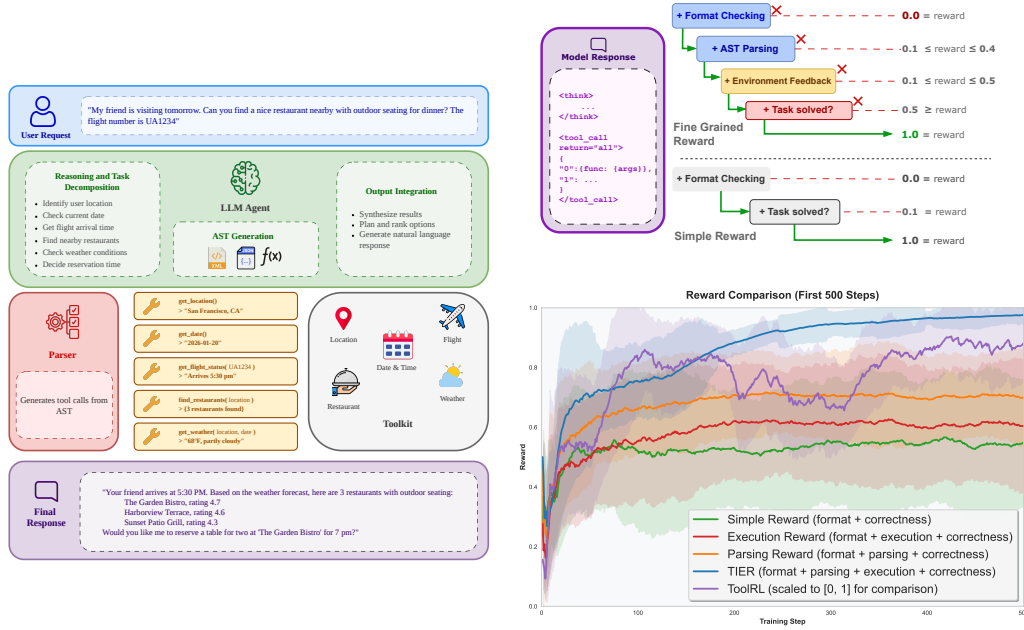


Figure 1: **Left:** Overview of TIER for tool-augmented LLM agents. The model generates structured tool calls as abstract syntax trees (ASTs), which are executed against external tools. Unlike trajectory-supervised approaches, rewards are computed directly from function schemas and execution outcomes. **Right (Top):** TIER provides fine-grained, multi-level supervision decomposing into format, schema, execution, and answer correctness. In contrast, simple outcome rewards offer only sparse binary feedback. **Right (Bottom):** Training curves under different reward designs. Only the full fine-grained TIER reward achieves stable convergence to high performance. Partial rewards plateau early, and trajectory-supervised rewards (ToolRL) exhibit high variance due to inconsistent supervision across alternative valid compositions. All training reward curves are exponentially weighted moving averages of per-step batch-mean reward logged over a 50-step rolling window (smoothing constant, $\alpha = 0.01$). The shaded region is a ± 1 rolling standard deviation. For visualization, we linearly rescale the rewards to $[0, 1]$

are vulnerable to changes in tool interfaces. These limits compound in compositional settings, where often multiple valid execution paths exist.

We propose TIER (Figure 1), a reward framework for tool learning that derives supervision directly from function schemas and runtime execution rather than reference trajectories. Each tool call is evaluated through a structured process that verifies format validity, schema adherence, execution success, and answer correctness, yielding dense, interpretable feedback at the sequence level. Because the verifier reads schemas directly at training time, supervision tracks schema changes automatically; unlike trajectory-supervised approaches [Qian et al., 2025], no reference trajectories must be regenerated when tools are added, modified, or deprecated. Any valid solution path receives appropriate credit, admitting multiple correct compositions that are typical of multi-step tasks. Empirically, TIER exceeds 90% accuracy across all composition depths on DepthBench and improves on external benchmarks including BFCL v3 and NestFUL.

Our contributions are as follows:

- We introduce TIER, a trajectory-invariant reward framework that leverages function schemas and execution feedback, eliminating the need for annotated reference trajectories.
- We introduce DepthBench, a depth-stratified benchmark spanning compositions of 1 to 6 steps, enabling systematic evaluation of compositional generalization.
- We show that reward components are complementary and not exchangeable, emphasizing that graded, multi-level supervision is essential for higher composition depths. We support our claims with gains on DepthBench and external benchmarks (BFCL v3, NestFUL).

We will publicly release DepthBench, the training data, TIER’s trained model checkpoints, and code upon acceptance, to support reproducibility and facilitate future research on compositional tool use.

2 TIER: Trajectory-Invariant Execution Rewards

TIER is built on a simple principle: rewards for tool-use trajectories should depend on *what the sequence accomplishes*, not *which path it follows*. Existing approaches only partially satisfy this requirement. Simple outcome-based rewards are trajectory-invariant, but provide no feedback on intermediate decisions, making credit assignment increasingly difficult as composition depth grows. In contrast, trajectory-supervised rewards offer graded feedback, but anchor it to a single reference trajectory, penalizing alternative yet valid solutions—a limitation that becomes more severe as the number of valid paths increases with depth. TIER bridges this gap by providing fine-grained, execution-grounded feedback derived directly from function schemas and runtime behavior, without relying on reference trajectories. The remainder of this section formalizes the notion of trajectory invariance and its implications, before introducing the four reward components that enable it.

2.1 Design principle: trajectory invariance

A reward function R is *trajectory-invariant* if its value on a candidate sequence y depends only on properties verifiable from y and the tool environment—format validity, schema adherence, execution success, and answer correctness—and not on y ’s similarity to any reference trajectory y^* .

This invariance has two practical consequences. On the space of valid solutions, all sequences satisfying the verifiable success, receive the maximum reward, regardless of which path was taken: alternative orderings of independent sub-goals, commutative operations, and functionally equivalent APIs all receive equal credit. On the space of invalid solutions, sequences are differentiated only by which verifiable properties they violate, providing a graded learning signal that does not depend on a reference trajectory. Trajectory-supervised rewards lack the first property—they arbitrarily privilege one valid path—while outcome-based rewards lack the second, collapsing the diversity of all error states into one.

A second benefit is operational: because rewards are derived from schemas and runtime execution, supervision tracks tool changes automatically, with no need to regenerate reference data when tools are added, modified, or deprecated.

TIER enforces trajectory invariance by construction: all reward components are evaluated against the current function schema and runtime behavior, with no reference to ground-truth trajectories. As shown in Section 4.2, violating this property leads to failure at higher composition depths: trajectory-supervised rewards collapse precisely because they penalize valid alternative solutions, while outcome-based rewards fail due to insufficient feedback.

2.2 Reward definition

TIER reward is computed once per generated tool-call sequence as

$$R_{\text{total}} = R_{\text{format}} + R_{\text{parse}} + R_{\text{exec}} + R_{\text{answer}}, \quad \text{with } R_{\text{total}} \text{ scaled to } [0, 1]$$

Each component verifies a distinct property of the full sequence—syntactic, structural, operational, and semantic—and is computed without reference to ground-truth trajectories. We represent tool sequences as structured ASTs that expose call ordering, argument bindings, and any nesting structure.

Format Validity. $R_{\text{format}} \in \{0, 1\}$ checks whether the full AST is well-formed and parsable. If this fails, all downstream rewards are zero.

Schema Adherence (Parsing). Decomposes as

$$R_{\text{parse}} = R_{\text{name}} + R_{\text{param}} + R_{\text{dtype}}, \quad R_{\text{parse}} \in [0, 3]$$

reflecting three granularities of schema correctness. Tool names are checked categorically

$$R_{\text{name}} = \begin{cases} 1 & \text{if all tool names are valid} \\ 0 & \text{otherwise} \end{cases}$$

since name validity determines which schema each call is checked against, any invalid API makes the sequence unverifiable and by extension invalid ($R_{\text{parse}} = 0$). Parameter and type correctness are graded by mismatch counts,

$$R_{\text{param/dtype}} = \text{clip}(1 - \lambda_p \cdot p, 0, 1),$$

where p is the total number of mismatches across all calls in the sequence, each call’s parameters checked against its own schema. We set $\lambda_p = 0.25$, balancing two failure modes: too large a coefficient eliminates partial credit for nearly-correct schemas, while too small a coefficient makes individual mismatches negligible.

Execution Success $R_{\text{exec}} \in \{0, 1\}$. Equals 1 iff every tool call executes successfully; otherwise 0. The all-or-nothing rule reflects the structure of compositional dependence: a failure at step k invalidates downstream calls that depend on its output, so a graded reward (e.g., fraction of successful calls) would credit work that produced no usable result. Treating chain-breaking failures as fully penalizing aligns the reward with the actual structure of compositional success.

Answer Correctness. $R_{\text{answer}} = 5 \cdot 1[\text{answer correct}] \in \{0, 5\}$. Equals 5 iff the final output matches the expected answer. We weight answer correctness substantially higher than the other components because lower weightings induce reward hacking — the model satisfies format, schema, and execution validity without producing correct answers, and training stalls before the answer correctness signal can take effect. The weighting prioritizes end-task success while preserving graded shaping signal from the lower-level components.

Sequence-level aggregation and trajectory invariance. All four components are evaluated over the full sequence rather than accumulated across per-step. Two consequences follow. First, all sequences producing the correct final answer and satisfying intermediate validity receive the maximum reward, regardless of length, ordering of independent sub-goals, or choice among functionally equivalent APIs. Second, sequences that fail are differentiated by which verifiable properties they violate providing graded correction signal that does not depend on any reference trajectory. TIER is therefore trajectory-invariant on the space of valid solutions, and informative on the space of invalid ones.

2.3 Reinforcement learning

We train models using a GRPO-style policy gradient objective with group-normalized advantages and token-level loss aggregation. Given a prompt x , we sample a set of trajectories $\mathcal{G}(x) = \{y_1, \dots, y_K\}$ from the current policy $y \sim \pi_\theta(\cdot | x)$ and execute each to obtain scalar rewards $R(x, y) \in [0, 1]$.

Group-normalized advantages. Advantages are computed at the trajectory level by normalizing rewards within each rollout group:

$$A(x, y) = \frac{R(x, y) - \mu_{\mathcal{G}(x)}}{\sigma_{\mathcal{G}(x)} + \varepsilon},$$

where $\mu_{\mathcal{G}(x)}$ and $\sigma_{\mathcal{G}(x)}$ are the mean and standard deviation of rewards over $\mathcal{G}(x)$, and $\varepsilon = 10^{-4}$.

Policy gradient objective. We optimize a PPO-style clipped surrogate objective at the token level. For a trajectory $y = (y_1, \dots, y_T)$:

$$\rho_t(\theta) = \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t | x, y_{<t})},$$

$$\mathcal{L}_{\text{PG}}(\theta) = -\frac{1}{\sum_{i \in \mathcal{B}} T_i} \sum_{(x, y) \in \mathcal{B}} \sum_{t=1}^{T_y} \min\left(\rho_t(\theta) A(x, y), \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) A(x, y)\right),$$

where $\mathcal{B}$ denotes the local batch and T_y is the number of active (non-padding) tokens in trajectory y . The loss is normalized by the total number of active tokens in the batch.

In our setup, we use a single update per rollout (`num_iterations` = 1), so $\pi_{\theta_{\text{old}}} = \pi_\theta$ at the update step and $\rho_t(\theta) = 1$. In this case, the clipped surrogate reduces in gradient to the standard REINFORCE estimator [Williams, 1992] $-A(x, y) \nabla_\theta \log \pi_\theta(y_t | x, y_{<t})$.

KL regularization. We apply per-token KL regularization to a reference policy π_{ref} using the k3 estimator [Schulman, 2017]:

$$\hat{\text{KL}}_t = \exp\left(\log \pi_{\text{ref}}(y_t | \cdot) - \log \pi_\theta(y_t | \cdot)\right) - \left(\log \pi_{\text{ref}}(y_t | \cdot) - \log \pi_\theta(y_t | \cdot)\right) - 1,$$

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PG}}(\theta) + \frac{\lambda_{\text{KL}}}{\sum_{i \in \mathcal{B}} T_i} \sum_{(x, y) \in \mathcal{B}} \sum_{t=1}^{T_y} \hat{\text{KL}}_t, \quad \text{with } \lambda_{\text{KL}} = 0.04$$

The reward formulation defined in Section 2.2 is agnostic to the optimization algorithm. We verify this by evaluating three policy optimization variants: GRPO-style, batch-normalized variants, and DAPO [Yu et al., 2025] under TIER, observing comparable performance (see Appendix F).

3 DepthBench: A depth-stratified benchmark for multi-step tool composition

3.1 Environment

DepthBench evaluates multi-step tool composition over a fully executable backend of 163 hand-crafted tools spanning three categories: information and utility functions, structured retrieval over synthetic datasets, and multi-step workflows. All tools execute deterministically, supporting both reproducible evaluation and the runtime grading needed to compute TIER’s execution and answer rewards. A rule-based parser validates schema adherence on each call, serving as the evaluation harness at test time and the R_{parse} verifier during training. To evaluate transfer beyond this controlled setting, we additionally benchmark on BFCL v3 [Patil et al., 2025] and NestFUL [Basu et al., 2025]

3.2 DepthBench design and evaluation protocol

DepthBench contains 1,710 queries stratified by composition depth from 0 to 6, where depth 0 denotes irrelevance cases in which no available tool can answer the query, and depths 1–6 denote single-call through six-call compositions. Multi-call compositions (depths 2–6) account for roughly half of the benchmark; the remainder consists of single-call queries and irrelevance cases. Composition structures include linear chains ($A \rightarrow B \rightarrow C$), funnels ($(A, B) \rightarrow C$), trees ($A \rightarrow (B, C)$), and order-agnostic conjunctive structures ($A \rightarrow (B \& C \& D) \rightarrow E$).

Although DepthBench is smaller than prior tool-use benchmarks, its construction is deliberately controlled: tool schemas are modeled after commonly-used APIs with executable backends implemented to simulate their behavior; queries are hand-crafted and verified against their intended composition structure. Comparable benchmarks make different trade-offs — BFCL v3 is LLM-generated and lacks an executable backend, while NestFUL provides 1,861 evaluation examples but is not stratified by depth — leaving DepthBench as the only setting we are aware of that combines realistic, executable tool semantics with depth-stratified evaluation. This stratification, absent from prior work([Patil et al., 2025, Basu et al., 2025, Zhong et al., 2025]), enables systematic analysis of compositional generalization rather than aggregate success.

Train–test splits use disjoint sets of target tools, with distractors also disjoint between splits, ensuring that no tool seen at training time appears at evaluation time in any role. Distractors are deliberately chosen to share schemas and output types with target tools, requiring models to distinguish functionally similar APIs rather than relying on superficial cues such as names or signature patterns. For multi-step evaluation, the dataset is split into 944 training and 766 validation examples, with tool schemas and sampled distractors provided at both training and evaluation time.

4 Experiment Results

4.1 Experimental setup

Model and Training. All experiments use Qwen3-8B as the base model. We train on a context length of 12,288 tokens with batch-normalized GRPO objective [Shao et al., 2024], regularized against a fixed reference policy (KL penalty $\lambda_{\text{KL}} = 0.04$). For each prompt, we sample $|\mathcal{G}| = 8$ trajectories and compute group-normalized advantages Section 2.3. Unless stated otherwise, the reward formulation from Section 2.2 is held fixed; Section 4.3 provides ablations over individual reward components. To isolate the effect of reward design from optimization, we additionally evaluate other RL objectives (GRPO, DAPO) under the same reward in Appendix F.

Baselines. We compare TIER against representative alternatives: (1) *Simple (outcome-based)*, a binary reward based only on format validity and final answer correctness, with no intermediate supervision; (2) *ToolRL (trajectory-supervised)* [Qian et al., 2025], which uses fine-grained rewards derived from matching reference trajectories; (3) *Qwen3-8B (zero-shot)*, the pretrained base model; (4) *Qwen3-8B (3-shot ICL)*, the base model with three in-context examples; and (5) *GPT-5 (3-shot ICL)*, a frontier reference under the same 3-shot protocol, included as a scale reference rather than a

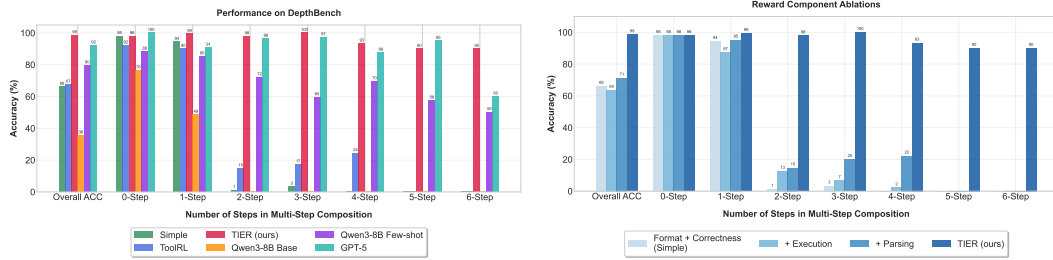


Figure 2: (a) Performance on DepthBench. (b) Reward component ablations under TIER

Table 1: Performance comparison on DepthBench, stratified by composition depth.

Reward Model	Method	Overall Acc	0-Step	1-Step	2-Step	3-Step	4-Step	5-Step	6-Step
Simple	RL	66.15	98.04	94.38	1.25	3.33	0.00	0.00	0.00
ToolRL	RL	67.49	92.16	90.00	14.74	17.33	24.44	0.00	0.00
TIER (ours)	RL	98.57	98.04	99.38	97.89	100.00	93.33	90.00	90.00
Qwen3-8B	Zero-shot	35.51	76.47	48.54	0.00	0.00	0.00	0.00	0.00
Qwen3-8B	3-shot ICL	79.56	88.24	85.25	72.12	59.55	69.57	57.50	50.25
GPT-5	3-shot ICL	92.16	98.02	90.11	96.36	97.28	87.59	95.00	60.00

head-to-head comparison given its substantially larger parameter count. All RL variants share the same model, optimizer, rollout budget, and hyperparameters; only the reward differs.

Evaluation Benchmarks. We evaluate on three benchmarks: (1) *DepthBench* (Section 3), is evaluated on the 766-example validation set, with results stratified by composition depth (0–6). (2) *BFCL* v3 [Patil et al., 2025], a real-world function-calling benchmark with different tools and evaluation protocols that measures out-of-distribution generalization; and (3) *NestFUL* [Basu et al., 2025], which evaluates nested and sequential API composition, where we report a controlled comparison against ToolRL under Qwen3-8B, and a broader comparison with tool-use models trained by other mechanisms. Finally, to evaluate whether DepthBench’s compact, composition-focused supervision is competitive with substantially larger tool-use corpora, we additionally train Qwen3-8B under TIER on RL adapted versions of ToolACE [Liu et al., 2025] and xLAM [Zhang et al., 2024] (Appendix E).

4.2 Multi-step composition requires trajectory invariance and execution awareness

We test the central claim that multi-step composition emerges only under TIER. We report DepthBench accuracy stratified by composition depth along with population-weighted average of per-depth accuracies in Table 1, Figure 2a.

Three observations follow. (1) TIER sustains high performance across all depths achieving $\geq 90\%$ accuracy at every composition depth, including 90% at depths 5 and 6 — the regime where depth-6 tasks introduce compound structures combining funnels and trees that no shallower depth contains.

(2) Outcome-based rewards fail beyond shallow composition. Simple-RL drops from 94.38% at one step to 1.25% at two steps and zero beyond four steps. Sparse binary signals provide no guidance for invalid solutions. Missing diversity in error states captured by the reward makes training over multi-step trajectories intractable. Notably this produces high-variance reward signals (Figure 1)

(3) Trajectory supervision collapses for a different reason. Although ToolRL provides intermediate feedback, it penalizes valid alternative compositions, reaching 0% accuracy at depths 5 and 6 where multiple valid paths are common. Training dynamics (Figure 1) clarify the mechanism: ToolRL’s reward exhibits high-variance oscillations because correct executions deviating from reference trajectories receive low reward, pushing the policy away from valid solutions. Notably the failure stems from a misspecified reward, not insufficient training.

RL versus prompting. Naive 3-shot ICL on Qwen3-8B (79.56%) outperforms both Simple-RL (66.15%) and ToolRL (67.49%), indicating that poorly-designed RL rewards can underperform no RL at all. ICL provides explicit per-structure demonstrations that bias the model toward valid compositions; in the absence of useful reward signal, RL training on Simple or ToolRL erodes this

Table 2: Effects of reward granularity as seen on validation set

Reward Model	Overall Acc	0-Step	1-Step	2-Step	3-Step	4-Step	5-Step	6-Step
Format + Correctness (Simple)	66.15	98.04	94.38	1.25	3.33	0.00	0.00	0.00
+ Execution (no Parsing)	63.58	98.04	87.29	12.63	6.67	2.22	0.00	0.00
+ Parsing (no Execution)	71.28	98.04	95.21	14.74	20.00	22.22	0.00	0.00
TIER (full reward)	98.57	98.04	99.38	97.89	100.00	93.33	90.00	90.00

Table 3: Comparison across different training objectives on BFCL.

Reward format	Overall	Non-Live AST	Live	Multiturn	Relevance	Irrelevance
TIER	68.92	85.94	81.96	39.12	77.78	87.65
Simple	66.30	85.00	78.72	34.00	77.78	83.56
ToolRL	37.27	84.10	43.98	0.38	94.44	0.81
Supervised Fine Tuning	61.47	84.12	79.52	19.25	61.11	90.33
Base (Qwen3-8B)	64.31	88.81	78.54	33.38	77.78	79.08

223 prior rather than building on it. TIER, by contrast, supplies reward signal aligned with composition
 224 structure, recovering and exceeding ICL’s gains across all depths.

225 4.3 Reward-component ablation

226 While Table 1 establishes that TIER enables multi-step composition, it does not isolate which
 227 components are necessary. We ablate the reward by removing components individually, holding
 228 the base model, optimizer, rollout budget, and RL algorithm fixed. Format and answer correctness
 229 are retained in all settings as the minimal baseline (identical to Simple-RL of Table 1). When a
 230 component is removed, its weight is redistributed and the total renormalized to $[0, 1]$ to control for
 231 scale. Results are shown in Table 2, Figure 2b.

232 **A sharp sufficiency threshold.** Partial rewards fail abruptly beyond shallow depths. Simple drops to
 233 0% beyond four steps; adding either execution or parsing yields modest improvement at intermediate
 234 depths but still collapses to 0% at depths 5 and 6. Only the full reward sustains $\geq 90\%$ across all
 235 depths. The transition from 0% to 90% at six steps under identical training represents a qualitative
 236 shift in training outcomes rather than gradual improvement: the components are not substitutes whose
 237 contributions sum incrementally, but a set whose union unlocks compositional behavior.

238 A counterintuitive result: adding execution reward without parsing decreases overall accuracy,
 239 with 1-step accuracy also dropping. We inspect failures by reviewing rollout logs at 100-step
 240 intervals throughout training. We observe a reward-hacking pattern: in the absence of schema-level
 241 discrimination from parsing, the model learns to route most queries to a small set of "safe" tools
 242 that reliably execute, accumulating execution reward while answering incorrectly. The weighting of
 243 answer correctness over execution prevents this collapse only when parsing supplies discrimination
 244 signal, but is insufficient on its own. This supports the rationale of designing parsing and execution
 245 as complementary components: parsing enforces structural validity (correct tool signature) and
 246 execution enforces functional validity (no runtime error). Neither suffices alone.

247 **Failure is due to signal saturation, not instability.** (Figure 1, right) show that all partial-reward
 248 configurations converge stably but plateau below the optimum. The model saturates the available
 249 signal; the signal itself lacks the structure needed for compositional behavior. Only the full reward
 250 reaches near-ceiling performance.

251 **Implications.** Multi-step composition requires concurrent supervision at three levels: syntactic
 252 (format), structural (schema), and functional (execution). Errors at each level produce different failure
 253 modes: format errors block parsing entirely, schema errors propagate to downstream calls, semantic
 254 errors corrupt outputs without raising syntactic flags. The fully decomposed reward, however,
 255 addresses error states at their origin. Compositional tool learning is therefore not a single objective
 256 but a multi-level credit-assignment problem requiring reward signals targeted at distinct failure modes.

Table 4: **NestFUL results.** (a) Training Qwen3-8B with TIER outperforms ToolRL. (b) TIER with Qwen3-8B compared against other tool learning methods. Exact Match Accuracy (EM Acc) is the ratio of test cases where all predicted APIs execute correctly and match the ground truth exactly. Baseline results are from [Basu et al., 2025].

(a) TIER vs. ToolRL		(b) Comparison with tool-use models					
Method	EM Acc		Hammer2.1-7B	TIER	xLAM-8x7b-fc-r	DeepSeek-V3	GPT-4o
ToolRL	0.476	# Param	7B	8B	46.7B	685B	UNK
TIER	0.684	EM Acc	0.08	0.75	0.03	0.60	0.60

257 4.4 Transfer to external benchmarks

258 **BFCL.** We evaluate generalization beyond the training environment on BFCL v3. All models are
 259 trained on a 1,240 example split from DepthBench and differ only in reward formulation. Additionally,
 260 we include a supervised fine-tuning (SFT) baseline and the pretrained base model for reference. To
 261 support evaluation on BFCL’s long-context tasks, we extend our model’s context with YaRN [Peng
 262 et al., 2023]. As shown in Table 3, TIER achieves the best overall performance (68.92%) and
 263 outperforms all baselines. ToolRL reaches comparable Non-Live AST accuracy (84.10), but collapses
 264 on Multiturn (0.38) and Irrelevance (0.81), where success requires adaptive decisions rather than
 265 reproducing a fixed trajectory. Simple rewards provide modest gains over the base model but remain
 266 below TIER, while SFT performs worse overall (61.47%) and degrades relative to the base model,
 267 reflecting overfitting to single reference trajectories. These results show that TIER rewards transfer
 268 better to real-world function-calling settings balancing both correctness and adaptability.

269 **NestFUL.** We evaluate on NestFUL under two protocols. In a controlled zero-shot comparison
 270 (Table 4(a)), TIER achieves a 0.684 exact-match accuracy versus ToolRL’s 0.476, confirming the
 271 advantage of TIER rewards. Under 3-shot ICL (Table 4(b)), our post-trained 8B model achieves 0.75,
 272 outperforming substantially larger models such as DeepSeek-V3 (685B, 0.60) and GPT-4o (0.60).
 273 In contrast, SFT-based models such as xLAM and Hammer2.1 achieve near-zero accuracies despite
 274 strong single-call accuracy, reinforcing that SFT alone is insufficient for compositional reasoning.

275 4.5 Intermediate representation for tool calls

276 Tool interaction requires a structured intermediate representation (IR) that can be deterministically
 277 parsed and executed. We evaluate JSON-based ASTs, XML-based ASTs, and direct generation with-
 278 out an IR under TIER (Table 5, Figure 3, Appendix A). JSON achieves the best overall performance
 279 and is therefore used as the default IR in all experiments. Since structured IRs for tool use are already
 280 well established in prior work, we defer implementation details to the appendix.

281 5 Related work

282 **Tool Use via Prompting and Supervised Learning.** Early systems rely on prompting-based methods
 283 such as Chain-of-Thought and ReAct [Wei et al., 2022, Yao et al., 2023], as well as engineered
 284 pipelines that interleave reasoning and tool calls [Paranjape et al., 2023, Zhuang et al., 2023, Song
 285 et al., 2023]. More recent work fine-tunes models on curated tool-use trajectories or schemas [Schick
 286 et al., 2023, Liu et al., 2024, Liang et al., 2024], improving robustness but requiring annotated
 287 intermediate actions and fixed tool interfaces. Several efforts train models to follow abstract function-
 288 calling schemas that generalize across tools via schema injection or retrieval [Qin et al., 2023, Du
 289 et al., 2025, Qi et al., 2024], enabling plug-and-play extensibility. However, these approaches rely on
 290 supervised objectives or exact-match evaluation and do not address learning from execution feedback.

291 **RL for Tool Use.** Recent work applies RL as a post-training paradigm for tool learning [Qian et al.,
 292 2025, Guo et al., 2025]. Outcome-based rewards rely solely on final task success, providing sparse
 293 feedback that does not assign credit across intermediate decisions. Concurrent work such as PORTool
 294 [Wu et al., 2025] addresses this by redistributing trajectory-level rewards using rollout-tree compar-
 295 isons, but the resulting signal remains path-dependent and tied to specific exploration trajectories.
 296 ToolRL [Qian et al., 2025] introduces fine-grained supervision, but derives it by matching model
 297 outputs to annotated reference trajectories, requiring curated data and penalizing valid alternative
 298 solutions. In contrast, TIER derives rewards directly from function schemas and runtime execution,
 299 providing fine-grained, trajectory-invariant supervision without relying on reference trajectories.

Multi-Step Tool Composition. Despite extensive work on tool learning, systematic evaluation and learning of multi-step composition remains limited. Existing benchmarks such as NestFUL [Basu et al., 2025] and ComplexFuncBench [Zhong et al., 2025] evaluate compositional tasks but do not stratify by composition depth. Prior RL approaches do not demonstrate reliable generalization across varying composition lengths. DepthBench addresses this gap directly by enabling per-depth accuracy measurement, which we show is essential for diagnosing reward failure modes. We demonstrate that TIER enables reliable composition where both outcome-based and trajectory-supervised rewards fail.

6 Conclusion and future work

We show that reward design is the primary bottleneck for multi-step tool composition in RL-trained language models. Outcome-based rewards fail at deeper compositions due to sparse credit assignment, while trajectory-supervised rewards penalize valid alternative solutions, a limitation that worsens as the number of feasible execution paths grows with composition depth. TIER addresses both issues by deriving rewards deterministically from function schemas and runtime execution, enabling fine-grained supervision that is invariant to the chosen solution path. Reward-component ablations further show that compositional tool use requires supervision at the syntactic, semantic, and execution levels simultaneously; removing any component substantially degrades performance. TIER achieves $\geq 90\%$ accuracy across all composition depths on DepthBench and generalizes consistently to BFCL v3 and NestFUL, demonstrating strong scalability and robustness for compositional tool use.

An important direction for future work is extending TIER from single-turn to multi-turn RL. By feeding model outputs from previous turns back into subsequent turns, the model can iteratively diagnose and correct failures. Since rewards are execution-grounded, both success and failure states can be incorporated automatically into the training signal, making it well-suited for scalable multi-turn training, where constructing such datasets is otherwise prohibitive. Another promising direction is applying TIER’s underlying RLVR principle to domains such as code generation and NL-to-SQL, where correctness can also be verified deterministically, enabling RL training without trajectory supervision. Finally, the depth stratification in DepthBench suggests a natural curriculum-learning strategy: training can begin with shallow compositions and progressively introduce deeper ones as the model’s capabilities improve. Such curricula may learn deeper compositions effectively.

7 Limitations

Our primary benchmark uses a synthetic, deterministic backend to enable controlled analysis of reward design, but real-world tool environments are stochastic and constrained, with latency, noisy outputs, partial failures, rate limits, monetary cost, and irreversible side effects. TIER assumes exploratory execution is cheap, repeatable, and safe, which may not hold in practice. Although BFCL v3 and NestFUL provide initial evidence of generalization, evaluating TIER in stochastic, cost-sensitive, and safety-critical settings remains future work. In addition, training is limited to a single-turn setting, while multi-turn evaluation is performed without explicit multi-turn RL training. Further we conduct all experiments at a single model scale using Qwen3-8B; although algorithm ablations in Appendix F suggest gains primarily arise from reward design, we do not study scaling behavior across model sizes. Reward weights are fixed based on preliminary tuning, and sensitivity to different weighting schemes across domains and composition depths is not explored. Finally, due to hardware constraints, training is limited to a 12,288-token context despite Qwen3-8B supporting substantially longer contexts via YaRN [Yang et al., 2025, Peng et al., 2023]. Consequently, the model is not exposed to long-context function-calling trajectories during training, and we exclude long-context benchmarks such as ComplexFuncBench [Zhong et al., 2025] after observing substantial degradation in preliminary experiments.

8 Impact Statement

This work advances post-training methods for large language models with an emphasis on reward design. The proposed approach reduces dependence on costly human annotation for tool-use training, which may broaden access to capable tool-using agents. As with any advance in autonomous agent capabilities, deployment in safety-critical or high-stakes environments requires appropriate guardrails, monitoring, and human oversight. We do not identify ethical concerns specific to this work beyond those generally associated with the development and deployment of language model agents.

References

- Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, et al. Nestful: A benchmark for evaluating llms on nested sequences of api calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33526–33535, 2025.
- Weihua Du, Hailei Gong, Zhan Ling, Kang Liu, Lingfeng Shen, Xuesong Yao, Yufei Xu, Dingyuan Shi, Yiming Yang, and Jiecao Chen. Generalizable end-to-end tool-use rl with synthetic codegym. *arXiv preprint arXiv:2509.17325*, 2025.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiusi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645 (8081):633–638, September 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z.
- Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, et al. Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis. *Intelligent Computing*, 3:0063, 2024.
- Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, Zezhong Wang, Yuxian Wang, Wu Ning, Yutai Hou, Bin Wang, Chuhan Wu, Xinzhi Wang, Yong Liu, Yasheng Wang, Duyu Tang, Dandan Tu, Lifeng Shang, Xin Jiang, Ruiming Tang, Defu Lian, Qun Liu, and Enhong Chen. Toolace: Winning the points of llm function calling, 2025.
- Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh RN, et al. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37:54463–54482, 2024.
- Zixian Ma, Weikai Huang, Jieyu Zhang, Tanmay Gupta, and Ranjay Krishna. m & m’s: A benchmark to evaluate tool-use for multi-step multi-modal tasks. In *European Conference on Computer Vision*, pages 18–34. Springer, 2024.
- Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. Art: Automatic multi-step reasoning and tool-use for large language models. *arXiv preprint arXiv:2303.09014*, 2023.

407 Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and
408 Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic
409 evaluation of large language models. In *Forty-second International Conference on Machine*
410 *Learning*, 2025.

411 Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window
412 extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.

413 Carl Qi, Yilin Wu, Lifan Yu, Haoyue Liu, Bowen Jiang, Xingyu Lin, and David Held. Learning
414 generalizable tool-use skills through trajectory generation. In *2024 IEEE/RSJ International*
415 *Conference on Intelligent Robots and Systems (IROS)*, pages 2847–2854. IEEE, 2024.

416 Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur,
417 and Heng Ji. Toolrl: Reward is all tool learning needs, 2025.

418 Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru
419 Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world
420 apis. *arXiv preprint arXiv:2307.16789*, 2023.

421 Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke
422 Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach
423 themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551,
424 2023.

425 John Schulman. Approximating kl divergence. <http://joschu.net/blog/kl-approx.html>,
426 2017.

427 Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang,
428 Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathemat-
429 ical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

430 Yifan Song, Weimin Xiong, Dawei Zhu, Wenhao Wu, Han Qian, Mingbo Song, Hailiang Huang,
431 Cheng Li, Ke Wang, Rong Yao, et al. Restgpt: Connecting large language models with real-world
432 restful apis. *arXiv preprint arXiv:2306.06624*, 2023.

433 Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny
434 Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in*
435 *neural information processing systems*, 35:24824–24837, 2022.

436 Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement
437 learning. *Machine Learning*, 8(3-4):229–256, 1992.

438 Feijie Wu, Weiwu Zhu, Yuxiang Zhang, Soumya Chatterjee, Jiarong Zhu, Fan Mo, Rodin Luo, and
439 Jing Gao. Portool: Tool-use llm training with rewarded tree. *arXiv preprint arXiv:2510.26020*,
440 2025.

441 An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang
442 Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu,
443 Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin
444 Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang,
445 Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui
446 Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang
447 Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger
448 Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan
449 Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.

450 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.
451 React: Synergizing reasoning and acting in language models, 2023.

452 Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong
453 Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi
454 Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi
455 Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying
456 Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source
457 llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

458 Yirong Zeng, Xiao Ding, Yutai Hou, Yuxian Wang, Li Du, Juyi Dai, Qiuyang Ding, Duyu Tang,
 459 Dandan Tu, Weiwen Liu, et al. Tool zero: Training tool-augmented llms via pure rl from scratch.
 460 In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 9135–9147,
 461 2025.

462 Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao
 463 Tan, Akshara Prabhakar, Haolin Chen, Zhiwei Liu, Yihao Feng, Tulika Awalgaonkar, Rithesh
 464 Murthy, Eric Hu, Zeyuan Chen, Ran Xu, Juan Carlos Niebles, Shelby Heinecke, Huan Wang,
 465 Silvio Savarese, and Caiming Xiong. xlam: A family of large action models to empower ai agent
 466 systems, 2024.

467 Lucen Zhong, Zhengxiao Du, Xiaohan Zhang, Haiyi Hu, and Jie Tang. Complexfuncbench: Exploring
 468 multi-step and constrained function calling under long-context scenario, 2025.

469 Yuchen Zhuang, Xiang Chen, Tong Yu, Saayan Mitra, Victor Bursztyn, Ryan A Rossi, Somdeb
 470 Sarkhel, and Chao Zhang. Toolchain*: Efficient action space navigation in large language models
 471 with a* search. *arXiv preprint arXiv:2310.13227*, 2023.

A AST-Based Intermediate Representations

We evaluate three strategies for function-call generation: JSON-based abstract syntax trees (ASTs), XML-based ASTs, and direct generation of tool calls without an intermediate representation.

These strategies impose different structural constraints and levels of syntactic flexibility, which affect training stability and execution correctness.

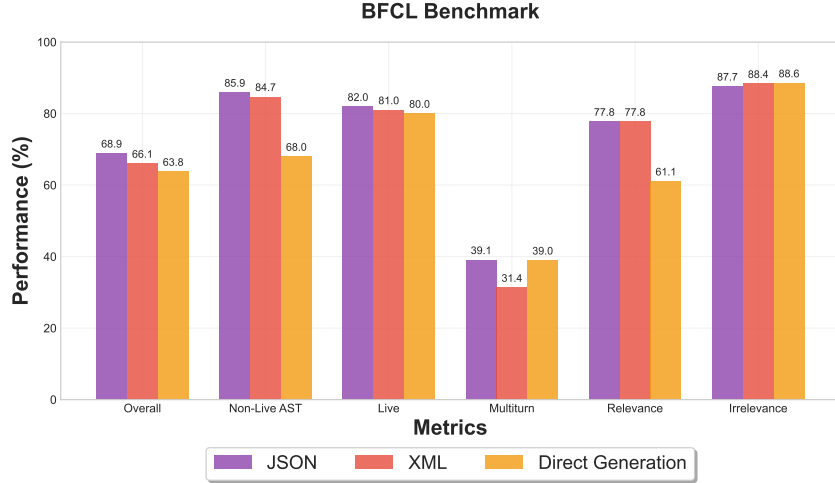


Figure 3: Performance of different intermediate representations under fine-grained rewards on BFCL v3.

Table 5: Comparison between generation strategies

IR format	Overall	Non-Live AST	Live	Multiturn	Relevance	Irrelevance
JSON	68.92	85.94	81.96	39.12	77.78	87.65
XML	66.06	84.73	81.03	31.37	77.78	88.37
Direct Generation	63.80	68.02	80.05	39.00	61.11	88.56

As shown in Table 5, JSON-based IRs achieve the strongest overall performance, outperforming both XML and direct generation across most metrics, including Non-Live AST, Live, Multiturn, and overall accuracy. These results are consistent with observations by [Ma et al., 2024, Patil et al., 2025].

Direct generation performs competitively on certain subsets—matching JSON on Multiturn (39.00 vs 39.12) and slightly outperforming both structured representations on Irrelevance (88.56). However, it degrades substantially in syntactic correctness (Non-Live AST: 68.02 vs 85.94 for JSON), leading to lower overall performance.

XML improves over direct generation in syntactic correctness and overall accuracy, but this advantage is not uniform across metrics: it underperforms direct generation on Multiturn (31.37 vs 39.00) and achieves comparable but not superior performance on Irrelevance.

Overall, structured IRs provide clear benefits for schema adherence and execution reliability, with JSON offering the best trade-off between structural consistency and downstream performance. Based on these results, we adopt JSON as the default intermediate representation in subsequent experiments.

A.1 XML-Based AST Format.

The XML-based IR represents a structured abstract syntax tree (AST) that explicitly encodes tool invocation order and inter-call dependencies:

AST

```
<api id="0">
  <name>API_NAME_0</name>
  <param name="P_0" type="string">VALUE_0</param>
  <param name="P_1" type="integer">VALUE_1</param>
</api>

<api id="1">
  <name>API_NAME_1</name>
  <param name="P_0">
    <response api_id="0" api_name="API_NAME_0" type="
      placeholder"/>
  </param>
</api>
```

493

494 The execution order is encoded through explicit API identifiers, while inter-call dependencies are
495 captured via response references, enabling deterministic execution of composed tool calls.

496 A.2 JSON-Based AST Format.

497 We additionally support a JSON-based AST format, where tool calls are represented as an ordered
498 mapping from call identifiers to function invocations:

AST

```
{
  "0": {
    "API_NAME_0": {
      "PARAM_NAME_0": VALUE_0,
      "PARAM_NAME_1": VALUE_1
    }
  },
  "1": {
    "API_NAME_1": {
      "PARAM_NAME_0": "API_RESPONSE_0"
    }
  }
}
```

499

500 In this format, dependencies between calls are expressed by referencing prior API responses. The
501 parser validates JSON well-formedness, function names, parameter schemas, and data types before
502 execution.

503 A.3 Direct Call List Format.

504 For simpler tool-use scenarios, we also support a direct call format, where tool invocations are
505 expressed as a comma-separated list within brackets:

AST

```
[
  API_NAME_0(PARAM_NAME_0=VALUE_0,
             PARAM_NAME_1=VALUE_1),
  API_NAME_1(PARAM_NAME_0=API_RESPONSE_0)
]
```

506

507 This format provides a lightweight representation for sequential tool calls without explicit AST
508 structure. The parser infers execution order from list position and resolves dependencies by matching
509 referenced outputs.

510 **B Function Composition Examples**

511 We illustrate function composition of varying depths using representative examples. Here, the number
512 of steps refers to the dependency depth of tool calls.

513 **B.1 Linear chaining**

514 Here, all calls are sequentially dependent, forming a chain $A \rightarrow B \rightarrow C$.

Prompt

Get all car listings from nearby dealerships.

515

AST

```
{
  "0": {
    "Get_current_location": {}
  },
  "1": {
    "Find_Dealerships_by_Location": {
      "location": "API_RESPONSE_0"
    }
  },
  "2": {
    "Get_Car_Listing_by_Dealerships": {
      "dealerships": "API_RESPONSE_1"
    }
  }
}
```

516

517 **B.2 Funnel Structure**

518 In this setting, intermediate calls are independent and jointly inform the final call, following the
519 structure $(A, B) \rightarrow C$.

Prompt

What is the current weather?

520

AST

```
{
  "0": {
    "Get_current_time": {}
  },
  "1": {
    "Get_current_location": {}
  },
  "2": {
    "Get_weather": {
      "time": "API_RESPONSE_0",
      "location": "API_RESPONSE_1"
    }
  }
}
```

521

522 B.3 Tree Structure

523 This structure requires a single call that is further branched out into multiple parallel calls $A \rightarrow$
524 (B, C) .

Prompt

What is the price of bitcoin on Jan, 11, 2026. Give me its value in British pounds, and in Euros?

525

AST

```
{
  "0": {
    "Get_Crypto_Price": {
      "ticker": "BTC",
      "price_time": "2026-01-11"
    }
  },
  "1": {
    "Currency_conversion_API": {
      "amount": "API_RESPONSE_0",
      "from_currency": "USD",
      "to_currency": "GBP",
      "price_time": "2026-01-11"
    }
  },
  "2": {
    "Currency_conversion_API": {
      "amount": "API_RESPONSE_0",
      "from_currency": "USD",
      "to_currency": "EUR",
      "price_time": "2026-01-11"
    }
  }
}
```

526

527 B.4 Conjunctive Constraint Composition

528 This scenario is a constraint-based composition over a shared entity, where multiple tool calls impose
529 independent predicates on the same object.

Prompt

Show me Japanese restaurants in San Diego that have ratings above 4.2.

Option A:

AST

```
{
  "0": {
    "Find_Restaurants_by_Location": {
      "location": "San Diego"
    }
  },
  "1": {
    "Filter_Restaurants_by_Cuisine": {
      "restaurants": "API_RESPONSE_0",
      "cuisine": "Japanese"
    }
  },
  "2": {
    "Filter_Restaurants_by_ratings": {
      "restaurants": "API_RESPONSE_1",
      "ratings": 4.2
    }
  }
}
```

Option B:

AST

```
{
  "0": {
    "Find_Restaurants_by_Location": {
      "location": "San Diego"
    }
  },
  "1": {
    "Filter_Restaurants_by_ratings": {
      "restaurants": "API_RESPONSE_0",
      "ratings": 4.2
    }
  },
  "2": {
    "Filter_Restaurants_by_Cuisine": {
      "restaurants": "API_RESPONSE_1",
      "cuisine": "Japanese"
    }
  }
}
```

Both syntaxes are valid since they could be applied in any order without changing the final result

B.5 Mixed structure

We could further combine multiple structures into one question that requires complex function composition

Prompt

Could you find Mexican restaurants near me that are open right now?

AST

```
{
  "0": {
    "Get_current_location": { }
  },
  "1": {
    "Find_Restaurants_by_Location": {
      "location": "API_RESPONSE_0"
    }
  },
  "2": {
    "Filter_Restaurants_by_Cuisine": {
      "restaurants": "API_RESPONSE_1",
      "cuisine": "Mexican"
    }
  },
  "3": {
    "Get_current_time": { }
  },
  "4": {
    "Filter_Restaurants_by_Opening_Hours": {
      "restaurants": "API_RESPONSE_2",
      "time": "API_RESPONSE_3"
    }
  }
}
```

C System Prompt

We show an example system prompt used for JSON AST format

System Prompt

Instruction

You are an intelligent agent capable of advanced tool use. You observe your environment, reason deeply about it, and use tools or APIs at your disposal to answer user queries. Queries may require single or multiple API calls. Multiple calls may be executed in parallel (returning all outputs) or chained sequentially (returning only the final output).

API Call Scenarios

1. ****Single API call****: The query requires execution of a single API whose output will be returned to the user. (one output)
2. ****Multiple Parallel API calls****: The query requires execution of a set of API calls simultaneously, and the outputs of all will be returned to the user. (all outputs)

3. ****Multiple Chained API calls****: The query requires APIs to be chained together, where the output of one API serves as the input to another, and only the output of the final API call in the chain is returned to the user. (one output)
4. ****Multi-step API calls (Single Query)****: A single query requires multiple sequential API calls. You respond with ONE API call at a time and wait for the environment to execute it and provide you with the tool response. If the tool response indicates that the problem is solved, return a NO-CALL response. If the tool call fails, retry it. If the tool call succeeds, proceed with the next call in the sequence.
5. ****Multi-turn Conversations****: An extended conversation where the user provides multiple queries across several turns. Each turn may require multi-step API calls (as described in scenario 4). You must retain and utilize contextual information from previous exchanges to handle follow-up queries effectively. After completing one user's request with a NO-CALL response, the user may follow up with related queries or entirely new tasks. Use information from previous turns when relevant to the current query.

Your Objectives

1. Determine whether the query can be solved using the APIs available to you. If not, inform the user accordingly with a NO-CALL response.
2. If the query can be solved, determine whether it requires a single API call, multiple parallel calls, multiple chained calls, or multi-step sequential calls.
3. Identify the appropriate API(s) from the available list and prepare a plan for solving the problem.
4. Determine the correct order of execution when chaining or sequencing is required.
5. Construct the final solution using the exact JSON format provided below.

Response Format

You must strictly respond in one of the following JSON formats:

Format 1: Single API Call

```
<think>
Explain your thought process to solve the question.
</think>
<tool_call return="one">
  {
    "0": {
      "API_NAME": {
        "PARAM_NAME_0": PARAM_VALUE_0,
        "PARAM_NAME_1": PARAM_VALUE_1
      }
    }
  }
</tool_call>
```

Format 2: Multiple Parallel API Calls

```
<think>
Explain your thought process to solve the question.
</think>
<tool_call return="all">
  {
```

```

        "0": {
            "API_NAME_0": {
                "PARAM_NAME_0": PARAM_VALUE_0
            }
        },
        "1": {
            "API_NAME_1": {
                "PARAM_NAME_0": PARAM_NAME_0
            }
        }
    }
}
</tool_call>

```

Format 3: Multiple Chained API Calls

```

<think>
Explain your thought process to solve the question.
</think>
<tool_call return="one">
    {
        "0": {
            "API_NAME_0": {
                "PARAM_NAME_0": PARAM_VALUE_0
            }
        },
        "1": {
            "API_NAME_1": {
                "PARAM_NAME_0": PARAM_NAME_0,
                "PARAM_NAME_1": "API_RESPONSE_0"
            }
        }
    }
}
</tool_call>

```

Format 4: Multi-step API Calls (One at a Time)

For multi-step scenarios, respond with ONLY ONE API call per response:

```

<think>
Explain the current step in your plan.
</think>
<tool_call return="one">
    {
        "0": {
            "API_NAME": {
                "PARAM_NAME_0": PARAM_VALUE_0
            }
        }
    }
}
</tool_call>

```

After receiving the tool response, proceed with the next step or return NO-CALL if complete.

Format 5: NO-CALL Response

Use this format when:

- The query cannot be solved using available APIs
- The problem has already been solved based on previous tool responses
- No further action is required

```

<think>

```

Explain why no API calls are required. Specify either: (1) the query cannot be solved using the available APIs, or (2) the problem has been solved based on the environment response.

</think>

<tool_call return="one">

{}

</tool_call>

Rules

1. ****API Usage****: Use only the provided APIs. Do not invent, assume, or reference APIs that are not explicitly listed.
2. ****Return Attribute****: Indicate whether the output of all APIs needs to be returned or only one output needs to be returned using the "return" attribute of the '<tool_call>' tag:
 - '<tool_call return="all">': All outputs will be returned to the user. Use this for parallel calls.
 - '<tool_call return="one">': Only the output of the final API call will be returned to the user. Use this for single, chained, or multi-step calls.
3. ****Parameters****: Include all required parameters in each API call. Optional parameters may be added if they improve the result or are necessary to fulfill the query.
4. ****API IDs****: Each API call must have a unique numeric string key. The keys should be zero-indexed (starting from "0").
5. ****ID Sequencing****: The keys used should be a sequence of increasing integers like "0", "1", "2", "3", etc. For chained calls, this sequence represents the order of execution.
6. ****JSON Structure****: The tool call JSON must be a dictionary where each key is a numeric string ID and each value is an object containing a single API name as key and its parameters object as value.
7. ****API Name****: Each API call object (e.g., '{"0": {...}}') must contain exactly one API name as the key, with its parameters as the value.
8. ****Parameter Format****: Parameters for each API should be provided as a JSON object with parameter names as keys and their values as the corresponding values.
9. ****Parameter Types****: API parameter values may be primitive types (boolean, integer, float, string), complex types (dict, array), or string placeholders referencing previous API responses.
10. ****Response References****: To reference the output of a previous API call in chained scenarios, use the string value "API_RESPONSE_X" where X is the numeric ID of the API whose output you want to reference.
11. ****Chaining APIs****: When chaining API calls together, use "API_RESPONSE_0", "API_RESPONSE_1", etc. as parameter values to pass the output from one API as input to another.
12. ****Multi-step Execution****: For multi-step scenarios (scenarios 4 and 5), respond with only ONE API call at a time (a JSON object with a single key "0"). Wait for the environment to execute it and provide feedback before proceeding with the next step.

13. ****Context Retention****: For multi-turn conversations (scenario 5), maintain awareness of previous exchanges and use that context when relevant to the current query.

14. ****NO-CALL Format****: When no API calls are needed, respond with an empty JSON object `{}` inside the `<tool_call return="one">` tag.

547

548 The available tool definitions are added at the end of this prompt dynamically. In addition to the
549 required tools we also add irrelevant and ambiguous tool definitions to elicit reasoning capabilities of
550 the model.

551 D Traces from the trained model

Prompt

Make reservation at HTL_LONDON_PARK for 7 nights from August 1 to August 8, 2024 for 3 people. Guest Robert Wilson, phone 555-0567, email rob@vacation.com. Charge credit card 3782-822463-10005, billing address 789 Pine St, Los Angeles for Robert Wilson.

552

Model Response

<think>

Okay, let's tackle this user request. The user wants to make a hotel reservation at HTL_LONDON_PARK for 7 nights from August 1 to August 8, 2024, for 3 people. The guest details are Robert Wilson, phone 555-0567, email rob@vacation.com. They also want to charge a credit card: 3782-822463-10005, with billing address 789 Pine St, Los Angeles.

First, I need to check which API to use. The available APIs are GetDate, BookHotel, Doordash.place_order, and BeanAndBrewCoffeehouse.get_opening_hours. The task is about booking a hotel, so the BookHotel API is the right choice.

Looking at the parameters required for BookHotel: hotel_id, number_of_nights, check_details (with check_in and check_out dates), number_of_people, personal_details (name, phone, email), and billing_details (name, payment_method, and credit info if applicable).

The user provided all the necessary details. The check-in date is August 1, 2024, and check-out is August 8, 2024. The number of nights is 7, which matches the date difference. The hotel_id is HTL_LONDON_PARK. Number of people is 3. Personal details include Robert Wilson, phone 555-0567, and email rob@vacation.com. Billing details require the credit card info and address. The payment method is credit, so the credit parameters need to be included.

Wait, the credit card number is given as 3782-822463-10005. The API might require it without hyphens, but the user provided it with hyphens. However, the parameter type is a string, so including the hyphens should be okay. The billing address is 789 Pine St, Los Angeles, and the billing name is Robert Wilson.

I need to structure the parameters correctly. The check_details should have check_in as "2024-08-01" and check_out as "2024-08-08". The number_of_nights is 7. The personal_details name is Robert Wilson, phone_number is 555-0567, and email is rob@vacation.com. The billing_details name is Robert Wilson, payment_method is "credit", and the credit object includes address, card_number, and city. The city is Los Angeles.

553

Wait, the billing address is 789 Pine St, Los Angeles. So the address field in the credit object should be "789 Pine St, Los Angeles", and the city is "Los Angeles". That makes sense. The card number is 3782-822463-10005, which I'll input as-is since it's a string.

Now, verifying all required parameters are present. The hotel_id is provided, number_of_nights is 7, check_details with both dates, number_of_people is 3, personal_details with name and phone, and billing_details with all required fields. All required parameters are covered.

So, the BookHotel API call should include all these parameters. There's no need for other APIs here. The user isn't asking for dates, ordering food, or coffeehouse hours. So this single API call should handle the reservation.

</think>

```
<tool_call return="one">
{
  "0": {
    "BookHotel": {
      "hotel_id": "HTL_LONDON_PARK",
      "number_of_nights": 7,
      "check_details": {
        "check_in": "2024-08-01",
        "check_out": "2024-08-08"
      },
      "number_of_people": 3,
      "personal_details": {
        "name": "Robert Wilson",
        "phone_number": "555-0567",
        "email": "rob@vacation.com"
      },
      "billing_details": {
        "name": "Robert Wilson",
        "payment_method": "credit",
        "credit": {
          "address": "789 Pine St, Los Angeles",
          "card_number": "3782-822463-10005",
          "city": "Los Angeles"
        }
      }
    }
  }
}
</tool_call>
```

554

Prompt

Can you find open French restaurants in my vicinity?

555

Model Response

<think>

Okay, let's see. The user is asking for open French restaurants in their vicinity. First, I need to figure out what APIs to use here.

556

Hmm, the available APIs include `GetCurrentLocation`, which can get the user's location. Then there's `FindRestaurantsByLocation`, which uses a location parameter to find restaurants. But the user wants French cuisine, so maybe after getting the restaurants, I need to filter by cuisine using `FilterByCuisine`. Also, they want open restaurants, so I might need to check the opening hours with `FilterByOpeningHours`. But wait, the current time is needed for that. So first, get the current time with `GetTime`. Then, get the location, then find restaurants in that location, filter by French cuisine, then check which are open now. But the user might not have provided a specific time, so maybe the opening hours are checked against the current time. Let me outline the steps:

1. Get current time using `GetTime`.
2. Get current location using `GetCurrentLocation`.
3. Use `FindRestaurantsByLocation` with the location from step 2 to get all restaurants there.
4. Filter those restaurants to French cuisine using `FilterByCuisine`.
5. Then, filter the French restaurants by opening hours using `FilterByOpeningHours`, using the current time from step 1.

Wait, but the `FilterByOpeningHours` requires a specific time. So after getting the current time, I can pass that as the time parameter. So the steps are sequential. Each step depends on the previous one's output. So this would be a chained API call scenario. Let me check the parameters.

For example, `FindRestaurantsByLocation` needs a location, which comes from `GetCurrentLocation`. Then `FilterByCuisine` needs the cuisine (French) and the restaurants DataFrame from `FindRestaurantsByLocation`. Then `FilterByOpeningHours` needs the current time and the filtered restaurants from `FilterByCuisine`. So the order is important. So the first call is `GetCurrentLocation`, then `FindRestaurantsByLocation` with that location, then `FilterByCuisine` with `cuisine=French` and the restaurants from the previous step, then `FilterByOpeningHours` with time from `GetTime` and the restaurants from the previous filter.

Then, the final result would be the open French restaurants. But the user might not have specified a time, so the opening hours check is necessary. However, if the current time is not provided, the `FilterByOpeningHours` can't be done. Therefore, the steps need to be in that order. So the first API call should be `GetCurrentLocation` to get the location. Then `FindRestaurantsByLocation` with that location. Then `FilterByCuisine` for French. Then `FilterByOpeningHours` with the current time. But the current time needs to be obtained first. Wait, the `GetTime` API can be called first to get the current time. Then `GetCurrentLocation`. Then `FindRestaurantsByLocation`. Then `FilterByCuisine`. Then `FilterByOpeningHours` using the time from `GetTime`. So the order is: `GetTime`, `GetCurrentLocation`, `FindRestaurantsByLocation`, `FilterByCuisine`, `FilterByOpeningHours`. But the user's query is to find open French restaurants, so the final step would be `FilterByOpeningHours`.

However, the user might not have a specific time, so the current time is used. Therefore, the correct sequence is to first get the current time, then the location, then find restaurants in that location, filter by French cuisine, then filter those by opening hours using the current time. So the API calls would be chained in that order. Let me check the parameters again. For example, the `FilterByOpeningHours` requires the time and the restaurants DataFrame. So after getting the French restaurants, we pass them to `FilterByOpeningHours` with the current time. So the steps are sequential. Therefore, the first API call should be `GetTime` to get the current time. Then `GetCurrentLocation`. Then `FindRestaurantsByLocation` with the location. Then `FilterByCuisine` with `cuisine=French`. Then `FilterByOpeningHours` with the time from `GetTime` and the restaurants from the previous step. So the final answer would be the filtered restaurants that are open now. But the user's query is to find open French restaurants, so the last step is necessary. Therefore, the correct

approach is to chain these APIs in the order mentioned. So the first call is GetTime, then GetCurrentLocation, then FindRestaurantsByLocation, then FilterByCuisine, then FilterByOpeningHours. Each step’s output is used as input for the next. Therefore, the tool calls would be in sequence. Let me structure this as a multi-step API call scenario. The first step is to get the current time. Then the location. Then find restaurants by location. Then filter by cuisine. Then filter by opening hours. So the initial call would be GetTime, then GetCurrentLocation, and so on. But the user might not have specified a time, so the current time is necessary.

Therefore, the first API call is GetTime. Then, the next is GetCurrentLocation. Then, using the location from GetCurrentLocation, call FindRestaurantsByLocation. Then, filter those with FilterByCuisine for French. Then, use the current time from GetTime to filter by opening hours. So the final answer would be the list of open French restaurants. Therefore, the correct sequence of API calls is GetTime, GetCurrentLocation, FindRestaurantsByLocation, FilterByCuisine, FilterByOpeningHours. Each step depends on the previous one’s output. So the first tool call is GetTime.

</think>

```
<tool_call return="one">
  {
    "0": {
      "GetTime": {}
    },
    "1": {
      "GetCurrentLocation": {}
    },
    "2": {
      "FindRestaurantsByLocation": {
        "location": "API_RESPONSE_1"
      }
    },
    "3": {
      "FilterByCuisine": {
        "cuisine": "French",
        "restaurants": "API_RESPONSE_2"
      }
    },
    "4": {
      "FilterByOpeningHours": {
        "time": "API_RESPONSE_0",
        "restaurants": "API_RESPONSE_3"
      }
    }
  }
}</tool_call>
```

558

559 E Dataset Comparison and Analysis

560 E.1 Adapting Existing Tool-Use Datasets for TIER

561 DepthBench is deliberately compact: 1.7K examples over 163 tools, compared to ~10K examples
 562 over ~26K tools in ToolACE and ~60K examples over ~4K tools in xLAM. To assess whether
 563 this small, composition-focused corpus is competitive with established large-scale tool-use datasets,
 564 we train Qwen3-8B under TIER on adapted versions of both, using the same reward formulation,
 565 optimizer, and rollout budget as in Section 4.

Table 6: Comparison between sizes of DepthBench and ToolAce

DATASET (SIZE)	OVERALL	NON-LIVE AST	LIVE	MULTITURN	RELEVANCE	IRRELEVANCE
DEPTHBENCH (1240)	68.92	85.94	81.96	39.12	77.78	87.65
TOOLACE (10K)	64.66	85.90	82.81	24.12	61.11	91.01
XLAM (60K)	63.02	89.29	79.16	21.22	77.78	83.92

E.2 Adaptation procedure

ToolACE and xLAM are released as supervised fine-tuning corpora providing a single annotated trajectory per query, without an executable backend; neither natively supports execution-grounded rewards. We construct executable variants by parsing each function schema to recover its parameter set, declared types, and a required/optional partition. Because the two corpora encode this partition differently, we apply dataset-specific rules. ToolACE exposes an explicit required field alongside per-parameter defaults; we mark a parameter as required iff it appears in required, with the field taking precedence when a parameter both appears there and declares a default. xLAM exposes no required field, so we treat any parameter with a declared default as optional and all others as required.

A call passes parsing when every supplied argument belongs to the schema, all values type-check against their declarations, and every required parameter is present; optional parameters may be omitted. Parsed calls return a deterministic response that echoes the resolved arguments, with omitted optionals materialised to their declared defaults prior to echoing. Because the source corpora contain no golden tool outputs, this design isolates correctness at the parameter-passing layer; an omitted optional and the same optional passed at its default value yield identical responses by construction, so optional-parameter choices cannot influence downstream correctness through the execution layer.

We score a call as correct under a relaxed match against the released trajectory: required parameters must match the gold values exactly, while optional parameters are not scored. The criterion is strictly stronger than parsing success and strictly weaker than full trajectory string match — the latter would penalise superficial reordering or default-value rewrites that leave behaviour unchanged.

E.3 Scope

DepthBench additionally contains multi-turn dialogues in which composition spans across turns; the present work focuses on single-turn composition, and the multi-turn split is excluded from both training and evaluation. ToolACE additionally contains a multi-turn split. We restrict this work to single-turn composition and exclude multi-turn data from both training and evaluation; the same restriction applies to DepthBench’s multi-turn split.

E.4 Results

Table 6 reports BFCL v3 performance for models trained on each corpus under identical TIER supervision. Despite being roughly an order of magnitude smaller, the DepthBench-trained model achieves the strongest overall performance. The gap is concentrated on the Multi-Turn metric (39.1 vs. 24.1 and 21.2); single-turn metrics (Non-Live AST, Live) lie within a few points across the three corpora and do not separate them.

E.5 Analysis

Two dataset-level properties appear to drive the gap. First, xLAM contains frequent mismatches between released trajectories and the corresponding tool schemas — argument names, types, or values in the annotated trajectory do not match the declared signature. Under execution-grounded rewards, these inconsistencies penalize schema-conformant calls, because the schema and the trajectory disagree on what counts as correct. Second, ToolACE pairs a very large tool vocabulary ($\sim 26K$) with relatively few examples ($\sim 10K$), so most tools appear in at most one query. The resulting models invoke individual tools too rigidly to handle the argument variation present at evaluation time. Neither property is visible from single-call accuracy, which is why aggregate function-calling benchmarks fail to expose the gap that BFCL Multi-Turn surfaces.

F RL Algorithm Ablation

To verify that the gains reported in this paper are driven by reward design rather than the choice of optimization algorithm, we train three models using different policy optimization methods—Standard GRPO, Batch-Normalized GRPO, and DAPO—under identical reward formulation, base model (Qwen3-8B), rollout budget, and hyperparameters. The only variable across conditions is the optimization objective. Figure 4 shows training reward curves as exponentially-weighted moving averages of per-step batch-mean reward logged over a 50-step rolling window (smoothing constant, $\alpha = 0.01$). The shaded region is a ± 1 rolling standard deviation.

F.1 Training Dynamics

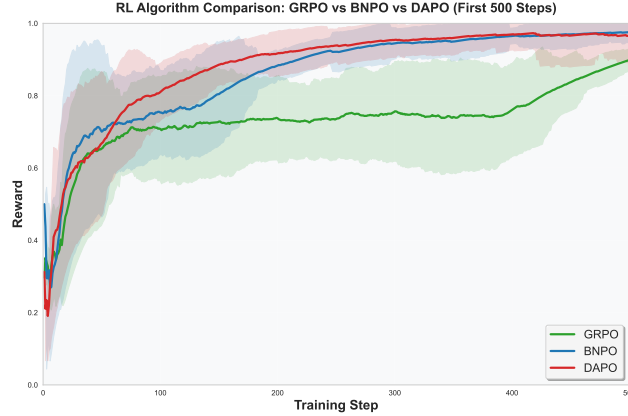


Figure 4: Training reward over the first 500 steps for three RL algorithms (Standard GRPO, Batch-Normalized GRPO, DAPO) under identical fine-grained, trajectory-invariant reward formulation. Shaded regions show ± 1 standard deviation. DAPO converges fastest, Batch-Normalized GRPO converges steadily to the highest level, and GRPO exhibits a delayed plateau-then-jump pattern. All three ultimately reach high reward, confirming that the reward design drives performance rather than the optimization algorithm.

DAPO converges fastest, reaching approximately 0.9 reward by step 100 and stabilizing with moderate variance. Batch-Normalized GRPO converges more gradually, reaching near-ceiling reward by step 200 and continuing to improve through step 500. Standard GRPO exhibits a distinct trajectory: it plateaus around 0.72–0.75 for an extended period (steps 100–350), followed by a delayed but sharp improvement after step 400, eventually approaching 0.9. This plateau-then-jump behavior suggests that GRPO requires a longer exploration phase before discovering effective compositional strategies, likely due to its reliance on group-relative comparisons. GRPO also shows higher variance throughout training, indicating less stable optimization compared to Batch-Normalized GRPO and DAPO.

Despite these differences in convergence behavior, all three algorithms ultimately reach high reward levels, demonstrating that the fine-grained reward signal is learnable across optimization objectives.

F.2 Evaluation Results

We report accuracy on the composition benchmark stratified by depth in Table 7.

All three algorithms achieve over 90% overall accuracy, confirming that the fine-grained TIER reward design is the primary driver of compositional capability rather than any specific optimization method.

However, the algorithms differ in their depth profiles. Batch-Normalized GRPO achieves the highest overall accuracy (98.56%) and the most consistent performance across depths (90–100%). Standard GRPO achieves comparable overall performance (97.17%) with particularly strong results at deeper compositions (97.78% at 5-step and 95.26% at 6-step), despite slower convergence during training.

Table 7: RL algorithm ablation on the composition benchmark. All configurations use fine-grained TIER rewards, base model, and rollout budget. Only the optimization objective differs.

Algorithm	Overall Acc	0-Step	1-Step	2-Step	3-Step	4-Step	5-Step	6-Step
Standard GRPO	97.17	98.04	98.10	95.00	96.47	98.93	97.78	95.26
Batch-Normalized GRPO	98.56	98.04	99.38	97.89	100.00	93.33	90.00	90.00
DAPO	93.21	100.00	94.79	98.95	76.00	88.89	100.00	70.00

636 DAPO exhibits the greatest variance across depths: while it achieves perfect accuracy at 0-step and
637 100% at 5-step, it drops to 76.00% at 3-step and 70.00% at 6-step.

638 This variance in DAPO aligns with its faster but less stable convergence—rapid policy commitment
639 yields strong performance on some compositions but weaker coverage of others. In contrast, GRPO’s
640 extended exploration phase appears to improve coverage across compositional patterns, leading to
641 more uniform depth performance.

642 Based on these results, we adopt Batch-Normalized GRPO as the default optimization method due to
643 its strong overall performance and consistency across depths. The key finding, however, is that *all*
644 *three algorithms enable multi-step composition when paired with fine-grained, trajectory-invariant,*
645 *execution rewards*. A reward design that yields near-zero accuracy at 5–6 steps under trajectory-
646 supervised training instead enables high multi-step performance across all optimization methods,
647 with GRPO and Batch-Normalized GRPO achieving consistently high accuracy and DAPO showing
648 more variable but still substantial gains.

649 G Training Compute

650 All training runs were conducted on an internal cluster using 2× NVIDIA A100 (80GB) GPUs per
651 run. Each run trained for 1,000 optimization steps and required approximately 20 wall-clock hours
652 (≈ 40 GPU-hours per run). In total, we trained well over 100 model configurations, corresponding to
653 roughly 4,000 GPU-hours of training compute. Each trained model was subsequently evaluated on
654 BFCL and DepthBench using a single A100 (80GB) GPU, amounting to approximately 1,680 GPU-
655 hours of evaluation compute. The full research project additionally incurred approximately 1,500
656 USD in OpenAI API costs across dataset evaluation, exploratory evaluations on ComplexFuncBench
657 that we ultimately excluded from the main results (see Section 7). Beyond the runs reported above,
658 additional GPU compute was spent on preliminary and discarded experiments that are not included
659 in this paper.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The claims made in the abstract and introduction match the experimental results in section 4

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Limitations of the work are discussed in section 7

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This work does not discuss any theoretical results. All empirical results are explained along with the experiment setup.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: This work discusses the TIER framework in detail for reproducibility. Training data, trained model checkpoints, and code will be publicly released upon acceptance.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

765 Question: Does the paper provide open access to the data and code, with sufficient instruc-
766 tions to faithfully reproduce the main experimental results, as described in supplemental
767 material?

768 Answer: [Yes]

769 Justification: Data and code will be publicly released upon acceptance

770 Guidelines:

- 771 • The answer [N/A] means that paper does not include experiments requiring code.
- 772 • Please see the NeurIPS code and data submission guidelines ([https://neurips.cc/](https://neurips.cc/public/guides/CodeSubmissionPolicy)
773 [public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 774 • While we encourage the release of code and data, we understand that this might not
775 be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not
776 including code, unless this is central to the contribution (e.g., for a new open-source
777 benchmark).
- 778 • The instructions should contain the exact command and environment needed to run to
779 reproduce the results. See the NeurIPS code and data submission guidelines ([https://](https://neurips.cc/public/guides/CodeSubmissionPolicy)
780 neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- 781 • The authors should provide instructions on data access and preparation, including how
782 to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- 783 • The authors should provide scripts to reproduce all experimental results for the new
784 proposed method and baselines. If only a subset of experiments are reproducible, they
785 should state which ones are omitted from the script and why.
- 786 • At submission time, to preserve anonymity, the authors should release anonymized
787 versions (if applicable).
- 788 • Providing as much information as possible in supplemental material (appended to the
789 paper) is recommended, but including URLs to data and code is permitted.

790 6. Experimental setting/details

791 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
792 rameters, how they were chosen, type of optimizer) necessary to understand the results?

793 Answer: [Yes]

794 Justification: Experiment setup and training environment are discussed in section 3, and
795 section 4

796 Guidelines:

- 797 • The answer [N/A] means that the paper does not include experiments.
- 798 • The experimental setting should be presented in the core of the paper to a level of detail
799 that is necessary to appreciate the results and make sense of them.
- 800 • The full details can be provided either with the code, in appendix, or as supplemental
801 material.

802 7. Experiment statistical significance

803 Question: Does the paper report error bars suitably and correctly defined or other appropriate
804 information about the statistical significance of the experiments?

805 Answer: [No]

806 Justification: Reporting statistical significance via multiple training runs is computationally
807 prohibitive for RL post-training of large language models. We therefore allocate compute to
808 extensive coverage of training conditions and ablations rather than repeated evaluation runs
809 per configuration. Each reported result corresponds to a fully trained model evaluated on
810 fixed benchmarks, ensuring deterministic evaluation given the same checkpoint. While this
811 setup does not allow estimation of variance across random seeds, the observed performance
812 differences are large and consistent across multiple benchmarks (DepthBench, BFCL v3,
813 and NestFUL), making the conclusions robust.

814 Guidelines:

- 815 • The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We discuss relevant details in appendix G.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: Yes

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Impact is discussed in section 8

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No such risks.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Our work uses several third-party models, datasets, and software components, each governed by their respective licenses. The base model Qwen3-8B is used under the Tongyi Qianwen License. The BFCL v3 benchmark is used under the Apache 2.0 License. Training and infrastructure rely on open-source libraries including TRL (Apache 2.0), DeepSpeed (MIT License), and PyTorch (BSD-style License), along with standard scientific Python packages such as NumPy and pandas (BSD-style licenses).

All newly created assets in this work, including DepthBench, training data, and model checkpoints, will be released under the CC-BY 4.0 license upon acceptance. We comply with all applicable third-party licenses and redistribute only artifacts permitted under their respective terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: All assets under license CC-BY 4.0. Assets will be publicly released upon acceptance.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: Does not apply

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

974 Answer: [N/A]
 975 Justification: Does not apply
 976 Guidelines:
 977 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
 978 with human subjects.
 979 • Depending on the country in which research is conducted, IRB approval (or equivalent)
 980 may be required for any human subjects research. If you obtained IRB approval, you
 981 should clearly state this in the paper.
 982 • We recognize that the procedures for this may vary significantly between institutions
 983 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
 984 guidelines for their institution.
 985 • For initial submissions, do not include any information that would break anonymity (if
 986 applicable), such as the institution conducting the review.
 987 **16. Declaration of LLM usage**
 988 Question: Does the paper describe the usage of LLMs if it is an important, original, or
 989 non-standard component of the core methods in this research? Note that if the LLM is used
 990 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
 991 scientific rigor, or originality of the research, declaration is not required.
 992 Answer: [N/A]
 993 Justification: LLMs were only used for writing, editing, or formatting purposes.
 994 Guidelines:
 995 • The answer [N/A] means that the core method development in this research does not
 996 involve LLMs as any important, original, or non-standard components.
 997 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
 998 be described.